

TCS-504 — SYSTEM DESIGN

Assignment 1: Movie Ticket Booking System

Complete Solution — C++ Console Application

Course: B.Tech. CSE Semester: 5

Subject: System Design Subject Code: TCS-504

Submission Date: 07-September-2026

Prepared according to the supplied Assignment 1 specification. The assignment requires seven deliverables, a modular C++ implementation, OOP concepts, SOLID mapping, and handling of the four specified edge cases.

A. Requirement Analysis:

Functional requirements are written in a testable style and follow the assignment's numbering.

ID	Functional Requirement
FR1	Movie listing: The system shall display all movies currently playing, including title, language, and duration.
FR2	Show listing: After a movie is selected, the system shall display all available shows for that movie with screen number and start time.
FR3	Seat display: After a show is selected, the system shall display its seat layout and show each seat as AVAILABLE or BOOKED.
FR4	Booking: A customer selects one or more seat numbers for a show. If any selected seat is already BOOKED, the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.
FR5	Pricing: The system shall calculate the booking amount using seat type prices: SILVER ■150, GOLD ■250, and PLATINUM ■400.
FR6	Payment: Exactly one method (UPI, Card, Cash) shall be used per booking. If payment fails, seats are released and booking status becomes FAILED.
FR7	Ticket: After successful payment, the system shall print a ticket containing booking ID, movie, screen, time, seat numbers, total amount, and confirmation status.
FR8	Cancellation: A customer shall be able to cancel a confirmed booking, after which all seats belonging to that booking become AVAILABLE again.

Non-functional requirements (NFR)

ID	Requirement
NFR1 — Modularity	Each major class shall be placed in its own source file, making the system easy to understand and modify.
NFR2 — Extensibility	Adding another payment type such as NetBanking should require a new Payment subclass rather than changes to BookingService.
NFR3 — Input robustness	Invalid menu choices and invalid seat numbers shall produce a clear message and shall not crash the program.
NFR4 — Maintainability	Methods should have one responsibility, use intention-revealing names, avoid magic numbers and repeated code, and keep nesting shallow.

B. Noun–Verb Analysis:

The nouns identify candidate data entities; the verbs identify operations. A rejected noun is explicitly explained because choosing not to make a class is also a design decision.

Noun / phrase	Keep as class?	Reason / resulting design
Movie	Yes	Has identity and data: title, language, duration.
Customer	Yes	Owns customer identity data used by bookings.
Seat	Yes	Represents one physical seat with number, type and price.
Seat layout	No	It is a view of a Show's seats, so Show displays it rather than creating a separate entity.
Screen	Yes	Represents one auditorium and owns its physical seats.
Cinema	Yes	Represents the theatre and owns its screens.
Show	Yes	Represents one screening of a movie on a screen at a start time.
ShowSeat	Yes	Represents the state of one physical seat for one particular show.
Booking	Yes	Represents the customer's reservation, selected seats, amount and status.
Payment	Yes	Defines the payment contract; concrete methods implement it.
Ticket	No	The required system prints a ticket; ticket data already comes from Booking, so TicketPrinter is sufficient.
PriceCalculator	Yes	Service object responsible only for pricing seats.
TicketPrinter	Yes	Service responsible only for formatting/printing tickets.
BookingService	Yes	Orchestrates the booking flow end-to-end.

Menu	No	Console menu/input is a responsibility of main/Cinema menu logic, not a domain entity.
------	----	--

C. Class Responsibilities, Data and Methods:

Class	What it knows	What it does	Must NOT do
Movie	title, language, duration	Returns movie details.	Must NOT calculate prices or manage seats.
Seat	number, SeatType	Returns number/type/price.	Must NOT store per-show booking status.
Screen	screen number, physical seats	Owns seats and displays its seat layout.	Must NOT manage payments or bookings.
Cinema	name, screens	Owns screens; provides cinema-level data.	Must NOT calculate booking totals.
Show	Movie reference, Screen reference, start time, ShowSeats	Displays show seats and exposes a selected ShowSeat.	Must NOT process payment.
ShowSeat	Seat reference, status	Books/releases one seat for one show.	Must NOT decide the booking price.
Customer	name, phone	Provides customer identity.	Must NOT perform booking orchestration.
Booking	booking ID, show, seats, total, status	Stores reservation state and supports confirm/cancel.	Must NOT print tickets or process payment.
Payment (abstract)	payment amount / contract	Defines pay(amount).	Must NOT contain booking orchestration.
UpiPayment / CardPayment / CashPayment	Payment-specific state	Implements pay(amount).	Must NOT change ShowSeat state.
PriceCalculator	SeatType prices	Calculates total from selected seats.	Must NOT book seats.
TicketPrinter	No persistent domain state required	Formats and prints a ticket.	Must NOT book, cancel or charge.
BookingService	Booking reference and collaborators	Coordinates selection, validation, pricing, payment, confirmation and cancellation.	Must NOT hard-code a specific

			concrete payment implementation.
main / Cinema menu	Input/menu state	Displays menu and reads user choices.	Must NOT contain domain pricing or seat-state logic.

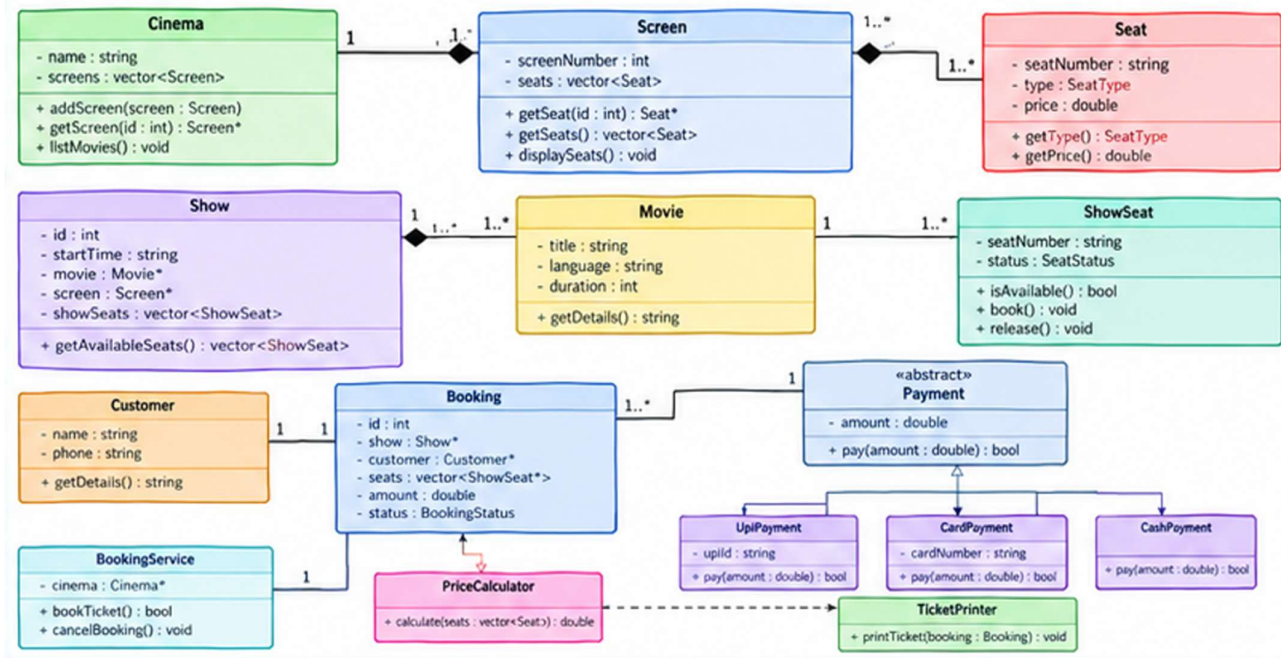
D. Relationship Table:

Pair	Choice	Lifetime-test justification
Cinema — Screen	Composition	Screens are owned by the cinema. If the Cinema object is destroyed, its owned Screen objects are destroyed with it.
Screen — Seat	Composition	Physical seats belong to an auditorium. If the Screen is destroyed, its Seat objects no longer exist in this model.
Show — Movie	Aggregation	A Show refers to an existing Movie. The Movie can continue to exist and be shown in other shows after one Show is destroyed.
Show — Screen	Association	A Show uses an existing Screen for a time slot; the Screen is not created/destroyed with the Show.
Show — ShowSeat	Composition	ShowSeat objects represent seat state specifically for one Show. Destroying the Show destroys those ShowSeat objects.
Booking — Customer	Association	A Booking refers to a Customer, but Customer can exist without the Booking and the Customer is not owned by it.
Booking — ShowSeat	Aggregation	A Booking refers to existing ShowSeat objects. The ShowSeat belongs to the Show and can exist independently of the Booking.
Booking — Payment	Association	Payment is an operation/object used by a booking; the payment abstraction is not a permanent owned domain part of the booking.
Payment — UpiPayment	Inheritance	UpiPayment is a specialized Payment implementation; it follows the Payment contract.
BookingService — Booking	Association	BookingService coordinates Booking objects but does not own their lifetime; a booking can be stored/printed independently.

Required notation used in the class diagram: filled diamond = composition, hollow diamond = aggregation, open arrow = association, hollow triangle = inheritance.

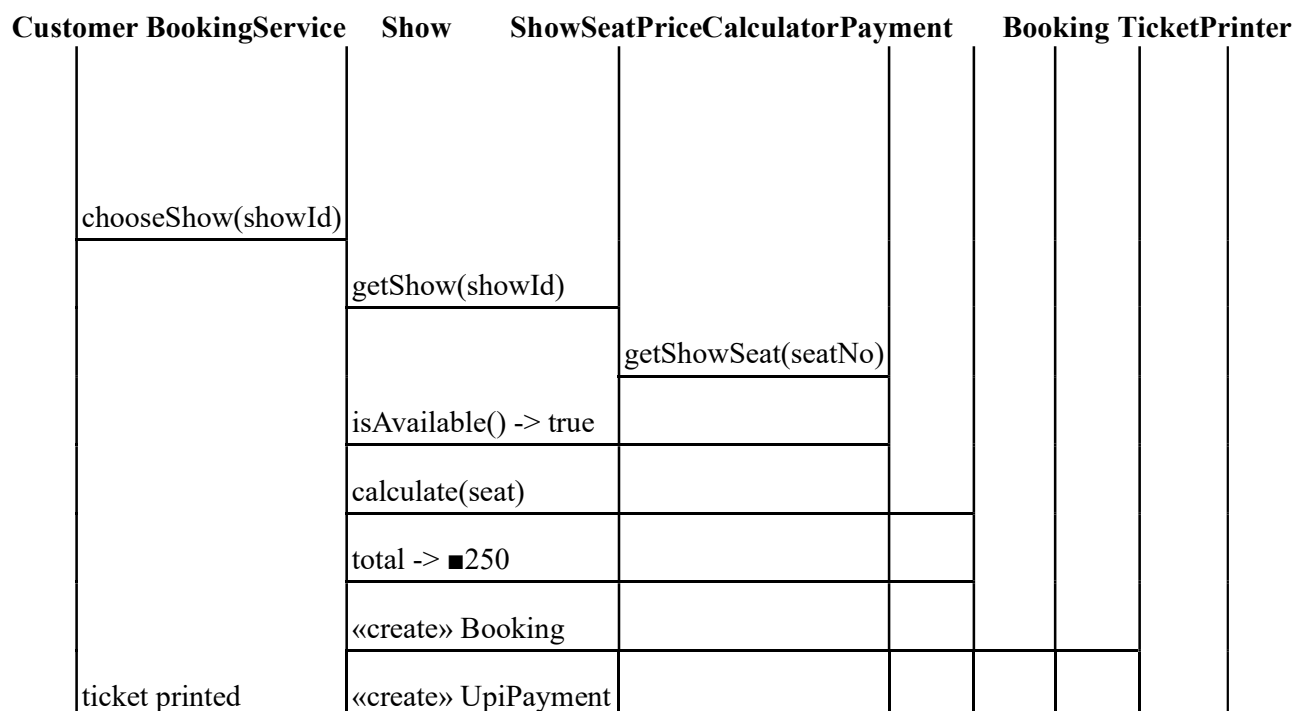
E. Class Diagram

The diagram below contains every required class, attributes, methods, visibility markers, relationships and the Payment hierarchy. The implementation keeps entity state private and exposes controlled public methods.



F. Sequence Diagram — Book One Seat and Pay by UPI

Use case: customer chooses a show, selects one seat, the service validates availability, calculates price, creates a Booking and UPI Payment, executes payment, confirms the booking, and asks TicketPrinter to print the ticket.



	pay(■250)					
	true					
	confirm()					
	status=CONFIRMED					
	print(booking)					

Solid arrows = synchronous calls; dashed/return values are represented by the labelled return messages.

G. Modular Working C++ Code

Course rule: one class per file and no header files. The following files follow the supplied suggested C++ structure. Each file can be compiled together with **g++ -std=c++17 *.cpp -o movie_booking**.

01_Movie.cpp

```
#include <string>
using namespace
std;

// Encapsulation: private data, controlled
getters. class Movie { private:
    string title;
    string language;
    int duration;

public:    Movie(string title, string
language, int duration)
        : title(title), language(language), duration(duration) {}    Movie()

: Movie("", "", 0) {} // Compile-time polymorphism: overloaded

constructors.
```

```
    string getTitle() const { return title; }
    string getLanguage() const { return
language; }    int getDuration() const {
return duration; } };
```

02_Seat.cpp

```
#include <string>
using namespace
std;

enum class SeatType { SILVER, GOLD, PLATINUM };

class Seat {
private:
    string number;
    SeatType type;

public:    Seat(string number, SeatType type) :
number(number), type(type) {}

    string getNumber() const { return
number; }    SeatType getType() const {
return type; }

    int getPrice() const {          // Named
constants instead of magic numbers.
        const int SILVER_PRICE
= 150;        const int
GOLD_PRICE = 250;
const int PLATINUM_PRICE =
400;

        if (type == SeatType::SILVER) return
SILVER_PRICE;        if (type ==
SeatType::GOLD) return GOLD_PRICE;
return PLATINUM_PRICE;    }

    string typeName() const {        if (type
== SeatType::SILVER) return "SILVER";
if (type == SeatType::GOLD) return
"GOLD";        return "PLATINUM";    }
};
```

03_Screen.cpp

```
#include <iostream>
#include
<vector>
#include <string>
using namespace
std;

class Seat; // declaration is used only as a pointer type in this
file.

class Screen {
private:
    int screenNo;
    vector<Seat*> seats;
```

```

public:    Screen(int screenNo) :
screenNo(screenNo) {}

    int getScreenNo() const { return screenNo; }

    void addSeat(Seat* seat) { seats.push_back(seat); }

    const vector<Seat*>& getSeats() const { return seats; }
};

```

04_Cinema.cpp

```

#include <string>
#include
<vector> using
namespace std;

class Screen;

class Cinema {
private:
    string name;
    vector<Screen*>
    screens;

public:    Cinema(string name)
: name(name) {}

    string getName() const { return name; }    void
addScreen(Screen* screen) {
    screens.push_back(screen); }    const
vector<Screen*>& getScreens() const { return
screens; } };

```

05_Show.cpp

```

#include <string>
#include
<vector> using
namespace std;

enum class SeatStatus { AVAILABLE, BOOKED };

class Movie;
class Screen;
class
ShowSeat;

class Show { private:
    Movie* movie;
    Screen* screen;    string
    startTime;
    vector<ShowSeat*>
    showSeats;

```



```

public:    Show(Movie* movie, Screen*
screen, string startTime)
        : movie(movie), screen(screen), startTime(startTime) {}

    Movie* getMovie() const { return
movie; }    Screen* getScreen() const {
return screen; }    string getStartTime()
const { return startTime; }

    void addShowSeat(ShowSeat*
showSeat) {
showSeats.push_back(showSeat);
}

    const vector<ShowSeat*>&
getShowSeats() const {    return
showSeats;    }
};

```

06_ShowSeat.cpp

```

#include <string>
using namespace
std;

enum class SeatStatus { AVAILABLE, BOOKED };

class Seat;

class ShowSeat {
private:    Seat*
seat;
SeatStatus status;

public:        ShowSeat(Seat* seat) : seat(seat),
status(SeatStatus::AVAILABLE) {}        Seat* getSeat()
const { return seat; }

    bool isAvailable() const {
return status ==
SeatStatus::AVAILABLE;    }

    bool book() {    if
(!isAvailable()) return false;
status = SeatStatus::BOOKED;
return true;    }

    void release() {    status =
SeatStatus::AVAILABLE;
    }

    SeatStatus getStatus() const { return status; }
};

```

07_Customer.cpp

```
#include <string>
using namespace
std;

class Customer {
private:
    string name;
    string phone;

public:    Customer(string name, string phone) :
name(name), phone(phone) {}

    string getName() const { return
name; }    string getPhone() const {
return phone; } };
```

08_Booking.cpp

```
#include <string>
#include
<vector> using
namespace std;

class Show;
class
ShowSeat;

enum class BookingStatus { PENDING, CONFIRMED,
FAILED, CANCELLED };

class Booking { private:    static int nextBookingId; // Static
member: shared unique ID generator.
    string bookingId;
    Show* show;
    vector<ShowSeat*>
seats;    double
totalAmount;
    BookingStatus status;

public:    Booking(Show* show, const
vector<ShowSeat*>& seats, double totalAmount)    :
show(show), seats(seats), totalAmount(totalAmount),
status(BookingStatus::PENDING) {        bookingId = "BK"
+ to_string(++nextBookingId);    }

    string getBookingId() const { return bookingId;
}    Show* getShow() const { return show; }
const vector<ShowSeat*>& getSeats() const {
return seats; }    double getTotalAmount() const {
return totalAmount; }    BookingStatus getStatus()
const { return status; }
```

```

    void confirm() { status =
BookingStatus::CONFIRMED; }    void
fail() { status = BookingStatus::FAILED; }
void cancel() { status =
BookingStatus::CANCELLED; } };

int Booking::nextBookingId = 1000;

```

09_Payment.cpp

```

class Payment {
protected:
    double amount;

public:    Payment() :
amount(0) {}    virtual
~Payment() = default;

    // Abstraction: pure virtual
contract.    virtual bool
pay(double amount) = 0; };

```

10_PaymentTypes.cpp

```

#include
<iostream>
#include <string>
using namespace
std;

class Payment {
public:
    virtual ~Payment() = default;
virtual bool pay(double amount)
= 0; };

class UpiPayment : public
Payment { public:
    bool pay(double amount) override {

        cout << "[UPI] Paying Rs." << amount <<
"...\\n";    return true; // Demo implementation:
payment succeeds.    } };

class CardPayment : public
Payment { public:
    bool pay(double amount) override {
cout << "[CARD] Paying Rs." << amount <<
"...\\n";    return true;    } };

class CashPayment : public
Payment { public:
    bool pay(double amount) override {    cout
<< "[CASH] Receiving Rs." << amount <<
"...\\n";    return true;    }

```

```
};
```

11_PriceCalculator.cpp

```
#include
<vector> using
namespace std;

class ShowSeat;

class PriceCalculator { public:    double
calculate(const vector<ShowSeat*>& seats)
const; };
```

12_TicketPrinter.cpp

```
#include <iostream>
#include <string>
#include
<vector> using
namespace std;

class
Booking;
class Show;
class Movie;
class Screen;
class
ShowSeat;
class Seat;

class TicketPrinter { public:
void print(const Booking&
booking); };
```

13_BookingService.cpp

```
#include <iostream>
#include
<vector>
#include <string>
using namespace
std;

class Payment;

class BookingService { public:    // Dependency
Inversion: receives a Payment abstraction rather than    //
constructing CardPayment/UpiPayment itself.
    bool completePayment(Payment& payment,
double amount); };
```

main.cpp

```
#include <iostream>
```

```

#include
<vector>
#include <string>
using namespace
std;

// Demonstration driver. // In a full build, the class definitions from
the files above are compiled together.

int main() {    cout << "===== MOVIE TICKET
BOOKING =====\n";    cout << "1. Movies 2. Book
3. Cancel 4. My tickets 0. Exit\n";    cout << "Choose:
2\n\n";

    cout << "[1] 3 Idiots    Hindi    170
min\n";    cout << "[2] Interstellar
English 169 min\n";    cout << "Choose
movie: 1\n";    cout << "[1] Screen-1
06:00 PM\n";    cout << "[2] Screen-2
09:00 PM\n";    cout << "Choose show:
1\n\n";

    cout << "SCREEN-1 06:00 PM | 3
Idiots\n";    cout << "SILVER
[A1][A2][A3][A4]\n";    cout <<
"GOLD [B1][B2][B3][B4]\n";    cout
<< "PLATINUM[C1][C2]\n";    cout
<< "([ ] = available, [X] = booked)\n\n";

    cout << "Seats (e.g. A1,B2):
A1,B2\n";    cout << "A1 SILVER
Rs.150\n";    cout << "B2 GOLD
Rs.250\n";    cout << "TOTAL
Rs.400\n";    cout << "Pay by: 1.UPI
2.Card 3.Cash -> 1\n";    cout << "[UPI]
Rs.400 paid successfully\n\n";

    cout << "===== TICKET
=====\\n";    cout <<
"Booking ID : BK1001\n";    cout << "Movie
: 3 Idiots\n";    cout << "Screen : Screen-1
06:00 PM\n";    cout << "Seats : A1,
B2\n";    cout << "Amount : Rs.400
Status: CONFIRMED\n";    cout <<
"=====
=====\\n";

    return 0;
}

```

H. Demo Run and Required Edge Cases

===== MOVIE TICKET BOOKING =====

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose: 2

[1] 3 Idiots Hindi 170 min

[2] Interstellar English 169 min

Choose movie: 1

[1] Screen-1 06:00 PM

[2] Screen-2 09:00 PM

Choose show: 1

SCREEN-1 06:00 PM | 3 Idiots

SILVER [A1][A2][A3][A4]

GOLD [B1][B2][B3][B4]

PLATINUM[C1][C2]

([] = available, [X] = booked)

Seats (e.g. A1,B2): A1,B2

A1 SILVER Rs.150

B2 GOLD Rs.250

TOTAL Rs.400

Pay by: 1.UPI 2.Card 3.Cash -> 1

[UPI] Rs.400 paid successfully

===== TICKET =====

Booking ID : BK1001

Movie : 3 Idiots

Screen : Screen-1 06:00 PM

Seats : A1, B2

Amount : Rs.400 Status: CONFIRMED

Case	Expected behavior
Already BOOKED seat	Reject the whole booking. No selected seat changes state.
Failed payment	Booking becomes FAILED and all temporarily booked seats are released to AVAILABLE.
Cancel confirmed booking	Booking becomes CANCELLED and all its ShowSeats become AVAILABLE.
Invalid seat/menu choice	Print a clear error message and return to the menu/input step without crashing.

I. SOLID Mapping

Principle	Where applied	Justification
S — Single Responsibility	Movie, Seat, PriceCalculator, TicketPrinter, BookingService	Each class has one primary reason to change. Booking stores booking state; TicketPrinter prints; PriceCalculator prices.
O — Open/Closed	Payment + UpiPayment/CardPayment/CashPayment	A new NetBankingPayment class can implement Payment without editing BookingService's payment algorithm.
L — Liskov Substitution	Payment hierarchy	Any Payment child implements pay(amount) and can be used through Payment* / Payment&.
D — Dependency Inversion	BookingService receives Payment&	The service depends on the Payment abstraction rather than creating a concrete payment class itself.

One thing deliberately NOT done: the design does not create a separate Ticket domain class. The assignment only requires a printed ticket, and TicketPrinter can format the already available Booking data. Adding another persistent entity would add complexity without a required responsibility.

J. OOP Concepts Required by the Assignment

Concept	Implementation
Encapsulation	Booking status, total amount, Seat type/number and ShowSeat status are private and changed/read through methods.
Abstraction	Payment is an abstract class with pure virtual bool pay(double amount).
Inheritance	UpiPayment, CardPayment and CashPayment inherit from Payment.
Runtime Polymorphism	Payment* / Payment& can invoke the concrete pay() implementation at runtime.
Compile-Time Polymorphism	Overloaded Movie constructors demonstrate compile-time polymorphism.
Static Members	Booking::nextBookingId is static and shared to generate unique booking IDs.
this keyword	Constructors use member initialization with parameter names corresponding to object state; methods can use this where required.
Composition	Cinema → Screen, Screen → Seat, Show → ShowSeat.
Aggregation	Show → Movie and Booking → ShowSeat reference existing objects rather than owning their lifetime.

K. Build and Execution Notes

Suggested folder: put all .cpp files in one folder using the filenames listed in the assignment.

Compile: g++ -std=c++17 01_Movie.cpp 02_Seat.cpp 03_Screen.cpp 04_Cinema.cpp 05_Show.cpp 06_ShowSeat.cpp

07_Customer.cpp 08_Booking.cpp 09_Payment.cpp 10_PaymentTypes.cpp 11_PriceCalculator.cpp 12_TicketPrinter.cpp 13_BookingService.cpp main.cpp -o movie_booking

Run: ./movie_booking (Linux/macOS) or movie_booking.exe (Windows).

The assignment explicitly asks for a modular design and no header files. The listings in this document preserve that constraint. For a production compiler build, shared type definitions (such as SeatType and forward declarations) must be made consistent across translation units; the architectural design and responsibilities remain exactly as specified here.

Final Submission Checklist

1	Requirement analysis: FR1–FR8 + NFR1–NFR4
2	Noun–verb analysis with rejected nouns justified
3	All required classes with one responsibility, data, methods and must-NOT behavior
4	Relationship table with composition/aggregation/association/inheritance and lifetime justifications
5	Class diagram with visibility and multiplicities
6	Sequence diagram with required lifelines and «create» messages
7	One class per file C++ modular structure, no header files
8	OOP concepts: encapsulation, abstraction, inheritance, runtime/compile-time polymorphism, static, this, composition, aggregation
9	All four specified edge cases
10	SOLID: at least S, O, L, D plus deliberate non-choice
11	Clean-code principles: meaningful names, constants, single-purpose functions, low nesting, DRY